

Analysis Report — Part A: Design Patterns in Web Applications

Laboratory Practice: Implementing Design Patterns (Singleton, Factory, MVC)

Student Name: _____ Student ID: _____
Course: Full-Stack Web Development Date: 14/9/2026

1. Singleton — Analysis Table (§7.1.4)

Analyzed project: 01-singleton (Simulated database connection manager in Next.js).

Question	Your Answer (Exact File & Line)
At exactly which line is it guaranteed that only one instance exists?	File: lib/DatabaseConnection.ts • Line 26: The constructor is declared <code>private constructor()</code>, blocking external code from instantiating it with 'new'. • Lines 33–38 (in <code>getInstance()</code>): Specifically at line 34, the condition <code>if (!DatabaseConnection.instance)</code> checks whether an instance already exists. If already instantiated, line 35 is skipped and line 37 returns the identical reference stored in the private static variable declared at line 18.
What would happen if the constructor were public?	If the 'private' modifier at line 26 were removed and the constructor were made public, any external module or consumer could freely invoke <code>'new DatabaseConnection()'</code>. This would completely break the Singleton guarantee: each call to 'new' would create a separate in-memory instance with a different 'connectionId', isolated query counters, and redundant connection handles, wasting server resources and eliminating centralized state.
Name a case in your own project where you would use Singleton.	In a centralized database connection client (such as Prisma Client, a MongoDB/Mongoose connection, or a PostgreSQL connection pool), or in an in-memory cache client (like Redis). The Singleton pattern ensures the entire application shares a single connection lifecycle, preventing exhaustion of the database server's connection pool limits.

2. Factory — Analysis Table (§7.2.4)

Analyzed project: 02-factory (Multi-channel notification factory: Email, SMS, Push).

Question	Your Answer (Exact File & Line)
Which line decides which concrete class is instantiated?	File: lib/notifications.ts

Question	Your Answer (Exact File & Line)
	The decision takes place in lines 55–69, inside <code>NotifierFactory.create(type)</code>. Specifically at line 56, the <code>switch (type)</code> branch evaluates the parameter: • Line 58: <code>return new EmailNotifier();</code> • Line 60: <code>return new SmsNotifier();</code> • Line 62: <code>return new PushNotifier();</code> • Line 67: throws an error if unsupported type is passed.
What would you need to change to add a WhatsApp channel?	Three targeted modifications are required: 1. In <code>lib/notifications.ts</code> (line 51): update type union: <code>export type NotificationType = "email" "sms" "push" "whatsapp";</code> 2. In <code>lib/notifications.ts</code>: define class <code>WhatsAppNotifier</code> implements <code>Notifier</code> { <code>readonly channel = 'whatsapp';</code> <code>send(to, message) { ... }</code> } and add a case to the factory: <code>case "whatsapp": return new WhatsAppNotifier();</code> 3. In the UI (<code>app/page.tsx</code>): add the select option <code><option value="whatsapp">WhatsApp</option></code>. Key Architectural Note: The API route controller (<code>app/api/factory/route.ts</code>) requires zero changes, adhering strictly to the Open/Closed Principle.
Name a case in your own project where you would use Factory.	In a payment gateway service for an e-commerce platform. A common interface 'PaymentGateway' defines 'processPayment(amount, details)'. The 'PaymentGatewayFactory.create(provider)' returns the concrete implementation ('StripeGateway', 'PayPalGateway', or 'MercadoPagoGateway') based on user selection, isolating payment vendor SDKs from business logic.

3. MVC (Model–View–Controller) — Analysis Table (§7.3.4)

Analyzed project: 03-mvc (Task list with strictly decoupled Model, View, and Controller layers).

Question	Your Answer (Exact File & Line)
Which file(s) correspond to each layer (M / V / C)?	• MODEL: <code>models/Task.ts</code> Defines the data contract (interface <code>Task</code>, lines 12–17), stores the in-memory array (lines 20–24), and implements pure domain rules (<code>findAll</code>, <code>create</code>, <code>toggle</code>, <code>remove</code>, lines 26–59). It has zero knowledge of HTTP or React. • CONTROLLER: <code>controllers/TaskController.ts</code> Validates inputs (e.g. required title on lines 21–24), coordinates with the Model, and formats HTTP response envelopes with status codes (200, 201, 400, 404). The API route handlers (<code>app/api/tasks/route.ts</code> and <code>app/api/tasks/[id]/route.ts</code>) serve as HTTP entry points delegating directly to this controller. • VIEW: <code>components/TaskForm.tsx</code>, <code>components/TaskList.tsx</code>, and <code>app/page.tsx</code>

Question	Your Answer (Exact File & Line)
	Responsible exclusively for visual presentation and notifying user interactions through callbacks (onCreate, onToggle, onDelete).
What would happen if TaskList.tsx fetched directly?	The fundamental Separation of Concerns (SoC) principle would be violated: 1. Tight Coupling: The presentation component would be inextricably bound to specific API routes (/api/tasks), destroying reusability in alternate contexts. 2. Impaired Testability: Unit and visual component testing would require mocking network endpoints instead of supplying test fixtures via props. 3. Brittleness: Any backend route or payload alteration would necessitate editing frontend UI rendering code.
Name a case in your own project where you would use MVC.	In a medical appointment scheduling or event ticket booking system: • Model (Appointment.ts): Enforces business invariants ensuring double-booking is prevented and manages database storage. • Controller (AppointmentController.ts): Validates input payloads, enforces authorization policies, and returns standardized responses. • View (CalendarGrid.tsx / BookingModal.tsx): Pure UI elements that display availability slots and dispatch user click events.

4. Reflection and Closing Questions (§11)

Individual answers for the group feedback and plenary discussion session:

1. Which of the three patterns did you find most intuitive, and which was hardest to justify? Why?

Answer: Singleton is the most intuitive pattern because its objective is immediate: guaranteeing a single global instance for expensive resources (such as database pools). Factory is the hardest to justify in small projects because when only one or two concrete classes exist that rarely change, adding abstract interfaces and factory creator classes introduces needless indirection and accidental complexity.

2. Think of a previous personal or school project. Would it have improved with one of these patterns? Which one, and why?

Answer: In a previous school project, database connections were often created anew across multiple handlers. Adopting Singleton for the database handle would have eliminated connection pool exhaustion and memory leaks. Furthermore, adopting MVC would have segregated presentation views from direct data store queries.

3. At what point could applying a pattern be counterproductive for a small project?

Answer: It becomes counterproductive when the structural overhead introduced by the pattern exceeds the inherent complexity of the application itself (violating KISS and YAGNI). Applying complex abstractions prematurely slows development velocity and obfuscates simple logic.

4. If you had to explain the difference between Factory and MVC to a classmate who has never programmed, what analogy would you use?

Answer: Factory is like a vending machine: you press a button specifying 'soda' or 'coffee' and it produces the concrete item without exposing its inner mechanisms. MVC is like an entire restaurant operation: the Model is the pantry holding ingredients and recipes, the Controller is the waiter taking orders and orchestrating the meal, and the View is the plated dish delivered to the diner's table.